Politecnico di Milano

Ingegneria del Software – a.a. 2008/09

Appello del 9 Febbraio 2010

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi ▪, Ghezzi ▪ , San Pietro ▪

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 7)

Scriverela specifica JML del seguente metodo statico:

public static ArrayList<Integer> sommeElementi(ArrayList<Integer> a)

Il metodo riceve come parametro un array list di interi, non vuoto e eventualmente contenente elementi ripetuti. Il metodo restituisce un altro array list che contiene:

- in prima posizione la somma di tutti gli elementi di a
- dalla seconda posizione in poi, gli elementi di a sommando eventuali ripetizioni. Ad esempio, se a contenesse: 5, 3, 7, 11, 3, 7, 3, l'array restituito dovrebbe contenere: 36, 5, 9, 14, 11 dove 9 è la somma delle tre occorrenze del numero 3 e 14 corrisponde ai due 7.

```
/* @requires a != null && a.size() > 0; */

/* @ensures \result != null && \resull.size() > 0 &&

Primo elemento di \result

    @ \result.get(0) == (\sum int i; i >=0 && i < a.size(); a.get(i)) &&

Tutti gli elementi di \result hanno senso

    @ (\forall int i; i > 0 && i < \result.size();
    @    (\exists int j; j >=0 && j < a.size();
    @.......\result.get(i) == a.get(j) * (\count int k; k >=0 && k
<a.size(); a.get(k) == a.get(j))))

Tutti gli elementi di a sono parte di \result

@ (\forall int i; i >= 0 && i < a.size();
    @    (\exists int j; j >0 && j < \result.size();
    @.......a.get(i) == \result.get(j) / (\count int k; k >=0 && k
<a.size(); a.get(k) == a.get(i))))
```

public static ArraList<Integer> sommeElementi(ArrayList<Integer> a)

Esercizio 2 (punti 13)

Un editor di figure geometriche consente di disegnare figure composte da nodi e da archi orientati che connettono i nodi. Ciascun nodo ha le coordinate x,y del suo centro.
Un arco orientato connette due nodi, detti sorgente e destinazione.
La specifica JML prevede due classi, Nodo e Arco, così definite.

```
public class Nodo {
        //@ensures (* costruisce un Nodo di centro (x,y)*);
        public Nodo (int  x, int y);
        //@ensures (*restituisce coord. x *);
        public /*@ pure @*/int  getX();
        //@ensures (*restituisce coord. y *);
        public /*@ pure @*/int getY();
        //@ensures  getX()==\old(getx())+dx  &&  getY()==\old(getY()
+dy);
        public void muovi(int  dx, int dy)
}

public /*@ pure @*/ class Arco {
        //@ensures (*costruisce un arco orientato da start a end*);
        public Arco(Nodo start, Nodo end);
        //@ensures (* restituisce l'origine dell'arco *);
        public Nodo origine();
        //@ensures (* restituisce la destinazione dell'arco *);
        public Nodo destinazione();
}
```

a) Si vuole descrivere, tramite una specifica, una classe Figura, che è costituita da un insieme di nodi e da un insieme di archi, ciascuno dei quali collega una coppia di nodi (non necessariamente distinti) della Figura. Completare, negli spazi bianchi, la specifica data di seguito. Si noti che:

   a1) Il metodo addNodo aggiunge un nodo n, a patto che n non abbia entrambe le coordinate uguali a quelle di nodi già presenti nella Figura.
   a2) Il metodo addArco(n1, n2) aggiunge, qualora non vi sia già un arco da n1 a n2, un nuovo arco da n1 a n2, e lo restituisce come risultato.
   a3) Il metodo arcoSeEsiste(sorg,dest) restituisce l'arco orientato che va dal nodo sorg al nodo dest, se l'arco esiste nella Figura, altrimenti restituisce il valore convenzionale null.
   a) Il numero massimo di nodi che si possono aggiungere a una figura è fissato al momento della costruzione della Figura (inizialmente vuota).

E’ possibile aggiungere, se ritenuto utile, opportune eccezioni ai metodi.

```
public class Figura {
/*@ensures (* restituisce la lista dei nodi della figura, in un ordine qualsiasi, senza ripetizioni*);
@*/
public ArrayList<Nodo> nodi()
/*@ensures (* restituisce la lista degli archi della figura, in un ordine qualsiasi,  senza ripetizioni*);
@*/
public ArrayList<Arco> archi()
/*@   requires maxsize>0;
      ensures (*Costruisce una Figura vuota che può contenere fino a                    maxSize
      nodi*); @*/
public Figura(int maxsize);
/*@   requires !(\exists Nodo n1; n!=n1 && nodi().contains(n1)
            n1.getX()==n.getX() && n.getY()==n1.getY()));
      ensures \old(archi()) == archi() && nodi().contains(n) &&
            (\forall Nodo n1; n1!=n;
                  \old(nodi().contains(n1) )<=> nodi().contains(n1);
@*/

public void addNodo(Nodo n);


//@requires n1!=null &&n2!=null;
//@ensures        \old(nodi()) == nodi() &&
       \result.origine()=n1 &&
      \result.destinazione() = n2 &&
      archi().contains(\result) &&
      (\forall Arco a; a!=\result;
            \old(archi().contains(a) ) <=> archi().contains(a));
public Arco addArco(Nodo n1, Nodo n2);


//@requires a!=null;
//@ensures \old(nodi()) == nodi() && !archi().contains(a) &&
      (\forall Arco a1; a1!=a;
            \old(archi().contains(a1) ) <=> archi().contains(a1);
public void deleteArco(Arco a);


//@requires sorg!=null && dest != null;
//@ensures (\exists Arco a; archi().contains(a);
```

```
                    a.sorgente() == sorg && a.destinazione()==dest)
        ? \result.sorgente()==sorg && \result().destinazione()==dest)
        : \result==null;
public /* pure */ Arco arcoSeEsiste (Nodo sorg, Nodo dest);
}
```

b) Descrivere un opportuno public invariant per la specifica di Figura. Argomentare che tale invariante è verificato dalla specifica o, nel caso non lo sia, spiegarne il motivo e mostrare che cosa dovrebbe essere modificato nelle specifiche date affinché sia verificato

L'invariante consta di due parti: nella prima si afferma che non vi sono due nodi che hanno le stesse coordinate; nella seconda che non vi sono due archi incidenti sugli stessi nodi.

```
public invariant
(\forall Nodo n; nodi().contains(n);
        !(\exists Nodo n1; n!=n1 && nodi().contains(n1);
                n1.getX()==n.getX() && n.getY()==n1.getY())) &&
(\forall Arco a; archi.contains(a);
        nodi().contains(a.origine()) && nodi().contains(a.destinazione()) &&
        !(exists Arco a1; a1!=a && archi().contains(a1);
                a1.origine()    ==    a.origine()    &&    a1.destinazione()==
a.destinazione())))
```

La specifica è organizzata in modo tale da verificare l'invariante, grazie alle postcondizioni di addNodo e di AddArco e deleteArco, ma *solo* sotto l'ipotesi che la classe Nodo sia immutabile. Tuttavia, la classe Nodo non è mutabile ed è quindi possibile modificare le coordinate di un nodo già inserito in Figura, potendo quindi rendere falsa la prima parte dell'invariante. Basterebbe quindi rendere immutabile la classe Nodo.

c) E’ data la seguente implementazione (parziale) di Figura.

```
public class Figura {
private ArrayList<Nodo> listaNodi;
private String [][] matriceArchi;
private int numNodi;
public Figura(int maxsize) {
        listaNodi = new ArrayList<Nodo>();
        matriceArchi = new Arco [maxSize] [maxSize] ;
        }
public void addNodo(Nodo n){
        if (numNodi == matriceArchi.size()) return;
        for (int i == 0; i< listaNodi.size() {
                n1= listaNodi.get(i);
                if (n1.getX() == n.getX() && n1.getY()== n.getY())
return;
        }
        listaNodi.add(n);
        numNodi++;
}
public Arco addArco(Nodo n1, Nodo n2){
        int sorg= listaNodi.indexOf(n1);
        int dest = listaNodi.indexOf(n2);
        matriceArchi [sorg][dest] = new Arco(n1,n2);
        return matriceArchi [sorg][dest];
}
public deleteArco(Arco a) {
        int sorg= listaNodi.indexOf(a.sorgente());
        int dest = listaNodi.indexOf(a.destinazione());
        matriceArchi [sorg][dest] = null;
}
```

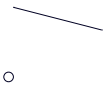
Scrivere l’invariante di rappresentazione della classe Figura e implementare il metodo arcoSeEsiste(…).

```
private invariant listaNodi != null && matriceArchi!= null &&
0 <= numNodi <= matrice Archi.size() &&
(\forall int i; 0<=i && i<matrice Archi.size();
        matrice Archi[i].size()== matriceArchi.size()) &&
(\forall int i; 0<=i && i<numNodi;
        (\forall int j; 0<=j && j<numNodi; matriceArchi [i][j] !=null
==>
                listaNodi.get(i) == matriceArchi[i][j].origine() &&
                listaNodi.get(j) == matriceArchi[i][j].destinazione()));

public Arco arcoSeEsiste(nodo n1, nodo n2) {
        i = listaNodi.indexOf(n1); j = listaNodi.indexOf(n2);
        if (i==-1 || j== -1) return null;
        return matriceArchi[i][j];
```

```
}
```

d) Si consideri l'implementazione parziale di una classe PoligonaleChiusa, riportata qui sotto. Una PoligonaleChiusa è una sequenza di almeno 2 nodi e altrettanti archi, che ritorna sempre al nodo iniziale. La figura seguente è un esempio con 5 nodi e 5 archi, in cui il nodo iniziale è marcato con un pallino.



```
public PoligonaleChiusa extends Figura {
        private Nodo primoNodo;
        private Nodo ultimoNodo;
        public PoligonaleChiusa(Nodo n1, Nodo n2, int maxSize) throws PoligonaleErrataException {
                super(maxSize);
                super.addNodo(n1);
                super.addNodo(n2);
                if (!nodi().contains(n1) !!  (!nodi().contains(n2)) throw new PoligonaleErrataException;
                super.addArco(new Arco(n1,n2));
                super.addArco(new Arco(n2,n1));
                primoNodo = n1;
                ultimoNodo = n2;
        }
//ogni nodo nuovo è aggiunto come ultimo nodo e collegato al primo:
        public void addNodo(Nodo n) {
                super.addNodo(n);
                if (!nodi().contains(n)) return;
                super.deleteArco(arcoSeEsiste(ultimoNodo, primoNodo));
                super.addArco(new Arco(ultimoNodo, n));
                super.addArco(new Arco(n, primoNodo));
                ultimo Nodo = n;
        }
        public Nodo getPrimoNodo() {return primoNodo;}
        public Nodo getUltimoNodo() {return ultimoNodo;}
…
}
```

In base all'implementazione data rispondere alla seguente domande:
d1) Il costruttore PoligonaleChiusa(…) deve prevedere delle precondizioni? Se si', quali?

E' sufficiente: n1 !=null && n2 != null && maxSize>=2

d2) E' possibile specificare la classe PoligonaleChiusa, completando eventualmente le parti mancanti, in modo che il principio di sostituzione sia verificato? Motivare accuratamente la risposta.

No, perche' ad esempio addArco non può aggiungere un arco fra due nodi qualora questo non esista: è vietato dalla definizione di PoligonaleChiusa (ossia dal suo invariante pubblico). Quindi si violerebbe o la regola dei metodi (se addArco "si rifiutasse" di aggiungere l'arco) o la regola delle proprietà (se addArco aggiungesse l'arco la proprietà di essere una PoligonaleChiusa non sarebbe verificata).
Analogamente con il metodo addNodo: nel momento in cui si aggiunge un nodo, si deve anche aggiungere anche un arco, eliminando nel contempo il vecchio arco dall'ultimo nodo al primo. La postcondizione di addNodo deve quindi modificare l'insieme degli archi, violando la postcondizione originale. Il costruttore invece va bene, in quanto i costruttori non si ereditano e la figura definita dal costruttore verifica l'invariante di Figura.

Per completezza, riportiamo qui anche l'invariante pubblico di PoligonaleChiusa:

```
/*@also public invariant archi().size() = nodi().size() &&
            arcoSeEsiste(ultimoNodo,primoNodo) != null &&
            (\forall Nodo n; nodi().contains(n); //ogni nodo ha 1 solo arco entrante e 1 uscente:
              (\num_of Nodo n1; nodi().contains(n1); ArcoSeEsiste(n,n1)!=null) ==1) &&
              (\num_of Nodo n2; nodi().contains(n2); ArcoSeEsiste(n2,n)!=null) ==1));

@*/
```

Esercizio 3 (punti 5)

Un’azienda è divisa in diverse sedi, dove lavorano dipendenti sia tecnici sia amministrativi. Ogni lavoratore percepisce uno stipendio, ma può anche venir licenziato, dare le dimissioni, oppure chiedere di cambiare sede. Gli amministrativi possono avere avanzamenti di carriera, mentre per i tecnici questo significa un cambiamento di mansione. Le due tipologie di dipendenti adottano modalità diverse per il calcolo dello stipendio. Ogni sede ha un budget proprio e provvede agli stipendi dei propri dipendenti in modo autonomo; l’azienda vuole solo poter sapere il numero totale dei propri dipendenti. Si definisca un opportuno diagramma delle classi per rappresentare l’organizzazione appena descritta evidenziando gli attributi e i metodi necessari.